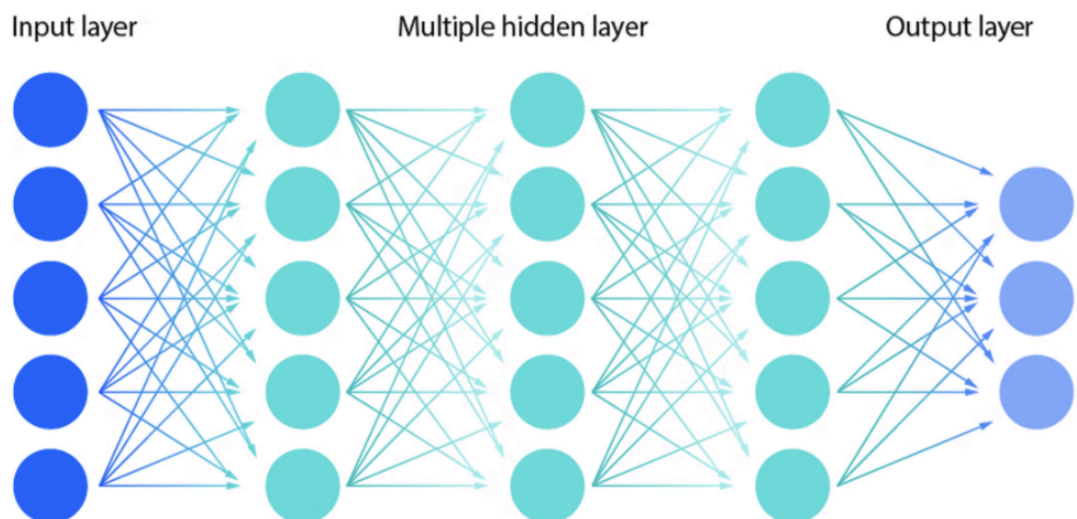


General Overview

Neural Networking is a predictive model, somewhat model the human brain and its structure, and is generally associated with forecasting, creating artificial media, and image recognition. Neural Networking involves Neurons being arranged in a way where there are three distinct layers associated with it.

- Input Layer: Comprises of the main raw features/data points (words are may also be a part of this, but we're not covering it)
- Multiple hidden layers: Linearly transforms the input raw values/features (discussed later in greater detail)
- Output layer: A non-linear activation function (tanh, sigmoid, ReLU) is then added to produce a final more meaningful output for humans.

Deep neural network



Example.

An email to a network and if it considers it spam or not:

The input layer recognises certain words in the email such as “dear”, “win”, “prize” etc. and uses those words as inputs.

The hidden layers capture these words and transform them linearly.

- Earlier neurons may capture the importance of each signal from the word, while later neurons capture the context and tone of the email. These layers then weight and transform the words/data points as well as the bias, leading into the final layer

The final layer then concludes the data, by using an activation function to calculate the output. This then links back to the original problem to determine if the email is considered spam or not. The email will be flagged as spam or not.

Neural Network Mathematical Training

$$z = \sum_{i=1}^n w_i x_i + b$$

$$a = \sigma(z)$$

Where:

- x_i = input feature,
- w_i = weight,
- b = bias,
- z = weighted sum (linear transformation),
- σ = activation function (nonlinear transformation),
- a = output,

The general mathematical formula for a **singular** neuron in a neural network looks like this. There may be thousands of these with differing weights and biases.

Essentially:

- Each node/input is multiplied with the weight added with the bias. This is then summed up all together, and creates z (think $y=mx+c$, but with multiple x values, while the y is added up).
- The weighted sum (z) is then put into the activation function to create an output

Note: The weighted sum will have different weights and biases, which will go into the activation function (non-linear function)

Finding the correct weights and bias

To ensure accuracy for the weights and bias, an algorithm called backpropagation is used

Backpropagation applies calculus to each individual neural network parameter to calculate changes on the overall error on the model's predictions.

The main goal of this is to observe how model weights and bias should be adjusted to guarantee the accuracy of the model.

This is done by using 4 steps:

Forward Pass

Neural networks passing the data through each layer of the network. Said above.

Cost/Error Function

Measures the difference between the predicted models' output and the true desired output.

Examples include Mean Squared Error (MSE) and Root Mean Squared Error (RMSE)

Backward Pass

The loss function is differentiated to identify the effect of an individual parameter on the overall error from a single input, and adjust the weights and bias of the neuron

Step by step process:

- Compute the Loss: Measure the difference between the predicted output and the true target value.
- Output Layer Gradient: Find how much the final layer's weights and biases contributed to the error.
- Chain Rule Application:
- Calculate local derivatives layer by layer, moving backward through activation functions. Parameter Updates: Adjust weights and biases using an optimizer like gradient descent to lower future errors